Laboratoire d'Intégration des Systèmes et des Technologies

DIASI / Département Intelligence Ambiante et Systèmes Interactifs

DIASI/SIR/25-0025/Rév. 0 **14/11/2025**

Rendu gaussien basé sur le ray tracing pour les réfractions et réflexions

Rapport de Stage

BAJI Jules

Commissariat à l'énergie atomique et aux énergies alternatives
Institut List | Département Intelligence Ambiante et Systèmes Intéractifs
Service Intérations et Réseaux | Laboratoire de Simulation Intéractive
CEA Saclay - Nanno-Innov - Bâtiment 861 - Point Courrier 173
91191 Gif-sur-Yvette Cedex
T. +33 (0)1 69 08 00 99
www-list.cea.fr

Établissement public à caractère industriel et commercial | RCS PARIS B 775 685 019

Tex/Texdisclaimer.tex 2009

TITRE : RAPPORT DE STAGE	DIASI/SIR/25-0025
AUTEUR : BAJI JULES	
UNITÉ : DIASI/SIR/LSI	Page 2
PROJET : RENDU GAUSSIEN BASÉ SUR LE RAY TRACING POUR LES RÉFRACTIONS ET RÉFLEXIONS	

NOMBRE DE PAGES : 33

RÉSUMÉ :
Le domaine de la reconstruction 3D photoréaliste a été profondément transformé par des approches neuronales récentes. Initialement, les champs de radiance neuronaux (NeRF) ont constitué une avancée majeure. Plus récemment, la méthode du *3D Gaussian Splatting* s'est imposée comme une nouvelle référence, offrant un compromis exceptionnel entre la qualité visuelle et la rapidité de rendu. Cette technique modélise une scène par un nuage de gaussiennes 3D, dont le rendu est effectué par projection (*splatting*).
Toutefois, cette méthode montre ses limites face à des scènes complexes, notamment celles qui présentent de fortes spécularités ou des surfaces réfractives et transparentes. Le présent stage a pour objectif d'explorer des solutions à ce problème. L'approche étudiée consistera à intégrer des techniques de lancer de rayons (*ray tracing*) et une modélisation plus fine des propriétés physiques des matériaux, tant pour l'entraînement du modèle que pour le rendu final. L'enjeu est d'améliorer la fidélité de la reconstruction pour des scènes contenant des surfaces et des volumes réfléchissants, réfractifs ou transparents.

MOTS-CLEFS : RECONSTRUCTION 3D, COMPUTER VISION, COMPUTER GRAPHICS, RADIANCE FIELDS, GAUSSIAN SPLATS, RAY TRACING

Rév.	Date	Rédacteur	Vérificateur	Émetteur	Approbateur	Pages modifiées
1		*Validation électronique*	*Validation électronique*	*Validation électronique*	*Validation électronique*	
0	14/11/2025	BAJI Jules	Chef de labo	Chef de projet	Chef de service	

LISTE DE DIFFUSION

DESTINATAIRE	SOCIÉTÉ	Adresse Email

Commissariat à l'énergie atomique et aux énergies alternatives
Institut List | Département Intelligence Ambiante et Systèmes Intéractifs
Service Intérations et Réseaux | Laboratoire de Simulation Intéractive
CEA Saclay - Nanno-Innov - Bâtiment 861 - Point Courrier 173
91191 Gif-sur-Yvette Cedex
T. +33 (0)1 69 08 00 99
www-list.cea.fr

Établissement public à caractère industriel et commercial | RCS PARIS B 775 685 019

TABLE DES MATIÈRES

TABLE DES MATIÈRES **4**

1 Remerciements **5**

2 Introduction **5**

3 Présentation du CEA **6**
3.1 Organisation et domaines d'activités 6
3.2 Historique du CEA 7
3.3 Présentation du Service d'Accueil 9
3.3.1 Direction de la Recherche Technologique (DRT) 9
3.3.2 Présentation de mon Institut : CEA-List 9
3.3.3 Présentation de mon laboratoire : 10

4 Travaux réalisés **10**
4.1 Approche 10
4.2 Première Partie : Etat de l'art 11
4.3 Seconde partie : Développement d'un prototype de lancer de rayons sur CPU 17
4.4 Validation des résultats par comparaison visuelle 23
4.5 Troisième Partie : Intégration des Gaussiennes Sphériques dans le moteur de rastérisation . . 26

5 Conclusion et Perspectives **32**

Références **33**

6 Annexes **33**

Commissariat à l'énergie atomique et aux énergies alternatives
Institut List | Département Intelligence Ambiante et Systèmes Intéractifs
Service Intérations et Réseaux | Laboratoire de Simulation Intéractive
CEA Saclay - Nano-Innov - Bâtiment 861 - Point Courrier 173
91191 Gif-sur-Yvette Cedex
T. +33 (0)1 69 08 00 99
www-list.cea.fr
Établissement public à caractère industriel et commercial | RCS PARIS B 775 685 019

1 Remerciements

Au terme de ce stage, je souhaite exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce projet.
Je tiens tout d'abord à remercier Maxime Maria, mon tuteur universitaire.
J'adresse mes plus vifs remerciements à mon maître de stage, Gilles Rougeron. Sa grande disponibilité, son sens de la pédagogie et sa connaissance profonde des sujets abordés ont été déterminants dans l'orientation et la démarche de mes travaux. Je le remercie pour sa confiance et sa bienveillance.
Je souhaite également remercier chaleureusement Hakim Chekirou pour son aide précieuse lors de la phase initiale de ce stage, ainsi que Mathis Boston, avec qui j'ai eu le plaisir de collaborer sur l'intégration des gaussiennes sphériques.
Je remercie également l'ensemble des membres du Laboratoire de Simulation Interactive (LSI) et le CEA dans son ensemble, pour m'avoir accueilli, pour l'excellente ambiance de travail et pour m'avoir fourni un cadre et des ressources exceptionnels.
Enfin, je remercie la Faculté des sciences et techniques de Limoges pour la formation qui m'a permis d'accéder à ce stage.

2 Introduction

La création de simulateurs haute-fidélité, capables d'intégrer de manière cohérente des objets synthétiques dans des environnements réels capturés, représente un enjeu majeur dans de nombreux domaines industriels. La méthode du 3D Gaussian Splatting (3DGS), par sa capacité à générer des rendus temps réel de très haute qualité à partir d'un simple ensemble d'images, s'est imposée comme une technologie de choix pour cette application.
Cependant, la méthode 3DGS originale atteint ses limites face à des scènes présentant des matériaux aux propriétés optiques complexes. La problématique au cœur de ce stage réside dans la restitution peu fidèle des surfaces présentant des effets spéculaires (reflets sur du métal ou du plastique verni), de la réfraction (distorsion à travers le verre) ou de la transparence. Le modèle d'apparence natif de 3DGS, basé sur les harmoniques sphériques, est en effet conçu pour représenter des phénomènes de basse fréquence et peine à modéliser la physique du transport de la lumière nécessaire pour ces matériaux.
Pour répondre à cette problématique, une solution en deux axes, inspirée par les avancées de l'état de l'art et notamment la méthode RayGauss, a été initialement envisagée : le remplacement du rendu par rastérisation par un pipeline de lancer de rayons, et l'enrichissement du modèle d'apparence par l'ajout de gaussiennes sphériques. La démarche de ce stage s'est articulée en deux phases distinctes. Une première phase d'exploration a consisté à développer un prototype de lancer de rayons sur CPU, afin de maîtriser les algorithmes fondamentaux. Par la suite, une analyse des coûts et des priorités du projet a conduit à une réorientation stratégique. L'effort a été concentré sur l'implémentation de la composante jugée la plus impactante et la plus efficace : les gaussiennes sphériques, intégrées directement au sein du moteur de rendu par rastérisation existant du laboratoire, `3dgs-studio`, et accélérées par GPU.
La contribution principale de ce stage est donc double : d'une part, le développement et la validation d'un prototype fonctionnel de lancer de rayons pour des scènes gaussiennes ; d'autre part, et surtout, l'implémentation réussie d'un modèle d'apparence avancé qui améliore significativement la qualité des reflets spéculaires avec un surcoût de calcul maîtrisé, démontrant ainsi la pertinence d'une approche pragmatique et ciblée.

Ce rapport est structuré en trois parties qui suivent le déroulement chronologique du projet. La première partie présente une analyse détaillée de l'état de l'art qui a permis de définir la solution cible. La seconde partie retrace le développement et la validation du prototype de lancer de rayons sur CPU. Enfin, la troisième partie expose l'implémentation des gaussiennes sphériques sur GPU, ainsi que l'analyse des résultats qualitatifs et quantitatifs qui valident la démarche adoptée.

3 Présentation du CEA

3.1 Organisation et domaines d'activités

Le Commissariat à l'Énergie Atomique et aux Énergies Alternatives (CEA) est un établissement public à caractère industriel et commercial créé en 1945 par le gouvernement de Charles de Gaulle. Originellement focalisé sur le développement de technologies nucléaires, le CEA a diversifié ses activités pour inclure les énergies alternatives, les sciences de la vie, et la recherche fondamentale. Aujourd'hui, il est un acteur majeur de la recherche, du développement et de l'innovation dans plusieurs secteurs clés :

— Énergies bas-carbone (nucléaire et renouvelables)
— Technologies pour l'information et la santé
— Très Grandes Infrastructures de Recherche (TGIR)
— Défense et sécurité globale

Monsieur François Jacq est l'administrateur général du CEA depuis 2018. Avec un budget de 5,8 milliards d'euros en 2022, le CEA emploie environ 21 000 personnes, réparties sur dix centres de recherche en France. Ces centres sont organisés en cinq pôles opérationnels, chacun dédié à des applications spécifiques :

— **D**irection de l'**E**nergie**s** (DES)
— **D**irection des **A**pplications **M**ilitaires (DAM)
— **D**irection de la **R**echerche **F**ondamentale (DRF)
— **D**irection de la **R**echerche **T**echnologique (DRT)

Évidemment, chaque pôle à sa spécificité et un domaine d'activité attribué. Présentons seulement les trois premières directions et examinons plus en détail la DRT, la direction dans laquelle je travaille :
La **D**irection de l'**E**nergies (DES) développe des systèmes énergétiques bas carbone, en couvrant l'énergie nucléaire et les énergies renouvelables. Elle se concentre sur la production d'énergie décarbonée, le fonctionnement technique des réseaux et la gestion des ressources. Elle soutient les technologies nucléaires actuelles et futures, développe des solutions pour le solaire et l'hydrogène, et supervise l'assainissement et la gestion des déchets des installations.
La **D**irection de la **R**echerche **F**ondamentale (DRF) du CEA mène des recherches de pointe en sciences de la matière et du vivant pour soutenir les transitions énergétique, climatique, numérique, ainsi que dans la médecine du futur, la défense et la sécurité.
La **D**irection des **A**pplications **M**ilitaires (DAM) du CEA a pour mission principale d'assurer la dissuasion nucléaire de la France. Cela inclut la conception, la fabrication, et le maintien en condition opérationnelle des armes nucléaires ainsi que la gestion des réacteurs nucléaires de propulsion navale. La DAM joue également un rôle crucial dans la lutte contre la prolifération nucléaire, le terrorisme et l'alerte aux tsunamis. Par ailleurs,

elle contribue à l'excellence de la recherche et à la compétitivité de l'industrie française en soutenant la défense conventionnelle et en intervenant dans des domaines variés de sécurité nationale.

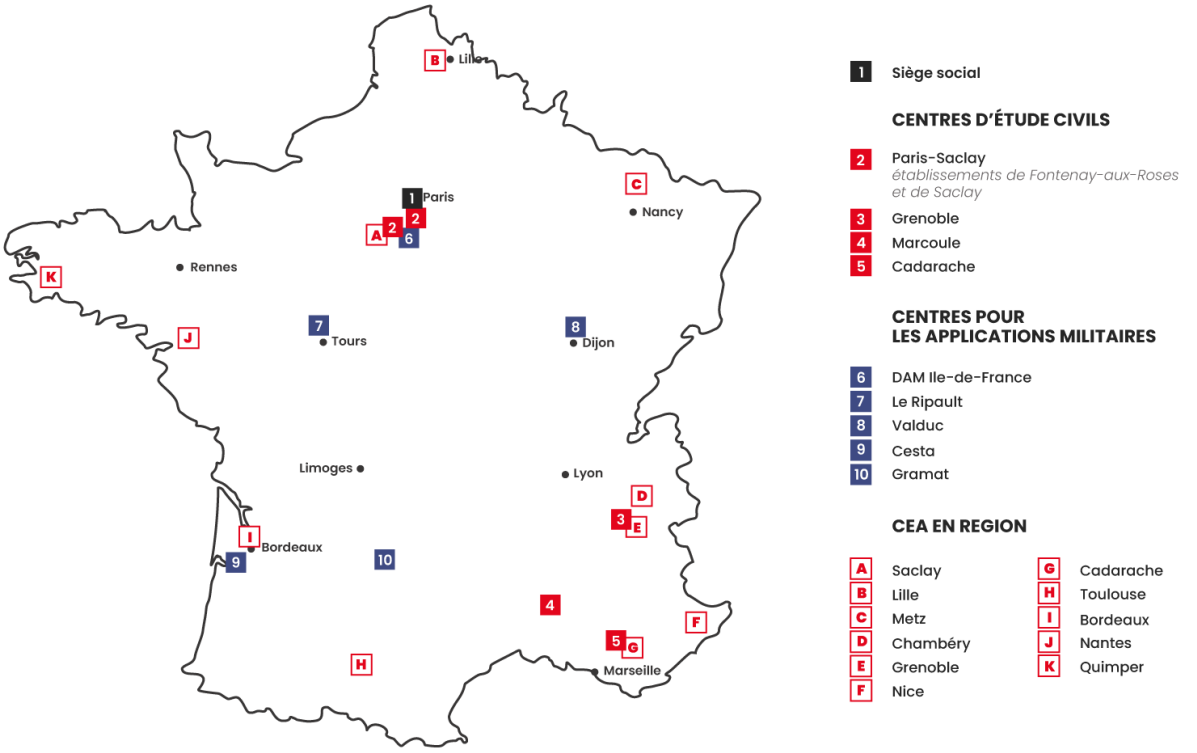


FIGURE 1 – Répartition des Centres CEA en France. Source : CEA

Le siège social du CEA est situé à Saclay, qui est également le plus grand centre de recherche multidisciplinaire d'Europe, employant environ 8 000 personnes.
Chiffres Clés :

- **6682** salariés CEA
- **676** doctorants et post-doctorants
- **327** CDD de recherche scientifique
- **672** demandes de brevets publiées en 2022
- **5559** salariés CEA à Saclay
- **708** salariés CEA à Fontenay-aux-Roses

3.2 Historique du CEA

La fondation du Commissariat à l'Énergie Atomique (CEA) en 1945 s'inscrit dans un contexte de reconstruction après la Seconde Guerre mondiale, où la France, malgré une situation difficile, possédait d'importantes compétences en physique atomique. À la demande du général Charles de Gaulle, le CEA est créé le 18 octobre

1945, avec pour mission de développer un pôle de recherche scientifique consacré à l'énergie atomique, visant à valoriser la France dans ce domaine.

Le premier site de recherche du CEA est établi au fort de Châtillon et inauguré en 1946. C'est sur ce site qu'a été développée, en 1948, la première pile atomique française utilisant de l'eau lourde.

Dès les années 1950, le CEA élargit ses infrastructures avec la construction de son premier centre d'études nucléaires à Saclay, opérationnel en 1952, qui marque une étape majeure pour la recherche nucléaire française. Le CEA y installe également son premier accélérateur de particules.

D'autres sites de recherche voient rapidement le jour : Marcoule en 1955, Grenoble en 1956, et Cadarache en 1959. Ces sites permettent au CEA de réaliser d'importantes avancées technologiques, notamment avec la construction des premiers réacteurs de type Uranium Naturel-Graphite-Gaz : G1 en 1956, G2 en 1959 et G3 en 1960 sur le site de Marcoule. Ces installations, ainsi que l'exploitation du plutonium, mènent à l'acquisition de l'arme atomique par la France en 1960.

Pour promouvoir la production d'électricité à partir de réacteurs nucléaires civils, une collaboration est instaurée entre EDF, le CEA et AREVA. Cette coopération aboutit au développement de la filière des réacteurs à neutrons rapides, avec des réalisations comme les réacteurs Phénix et Superphénix.

Organisation du CEA :

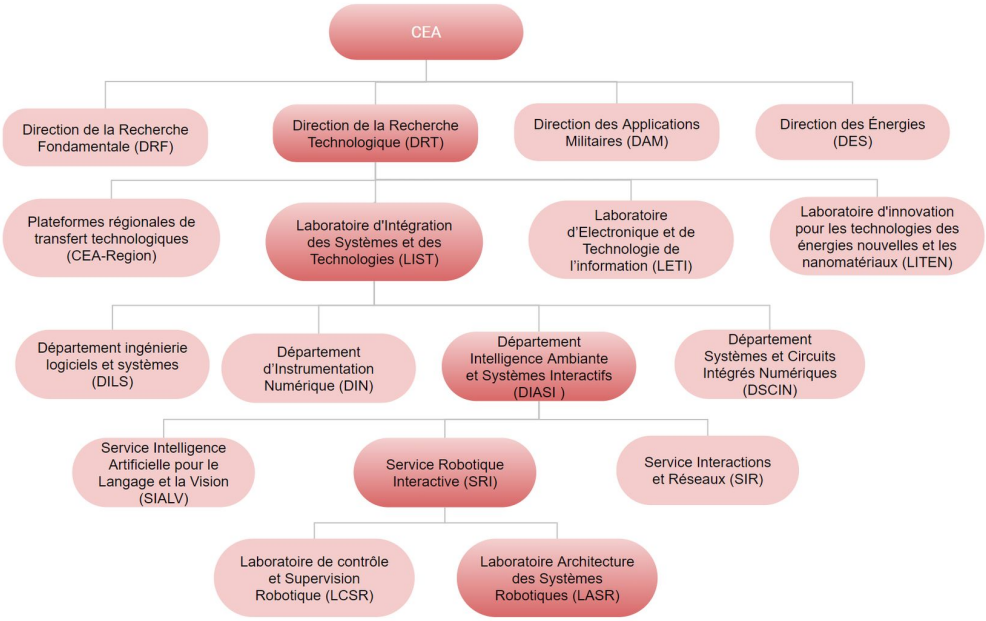


FIGURE 2 – Organigramme général du CEA

3.3 Présentation du Service d'Accueil

3.3.1 Direction de la Recherche Technologique (DRT)

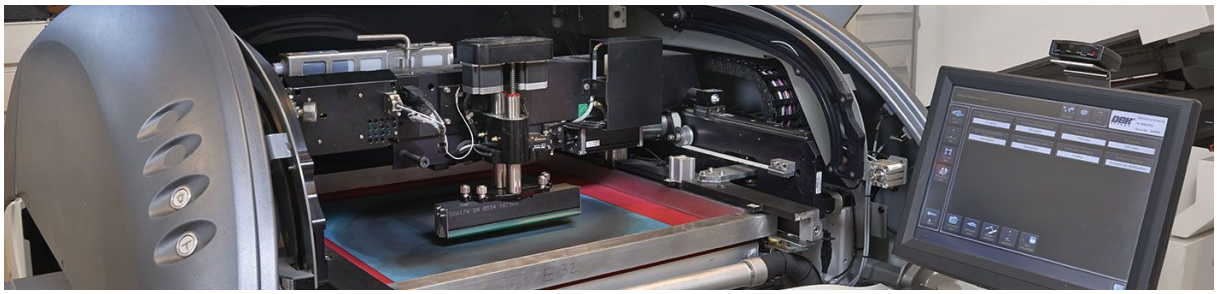


FIGURE 3 – Technologie de la pile à combustible. Source : CEA Tech

La Direction de la Recherche Technologique (DRT) ou le CEA Tech est le pôle de recherche technologique du CEA. Il est un acteur clé de l'innovation industrielle en France. Ce pôle est constitué de trois instituts principaux :

- LETI, spécialisé dans les micro et nanotechnologies
- LITEN, axé sur les nouvelles technologies de l'énergie et les nanomatériaux
- LIST, focalisé sur les systèmes numériques intelligents

CEA Tech emploie 4500 collaborateurs qui travaillent à développer et transférer des technologies génériques vers divers secteurs industriels, contribuant ainsi à la compétitivité des entreprises françaises.

CEA Tech se distingue par le développement de Technologies Clés Génériques (KETs), qui diffusent largement dans tous les secteurs industriels et sont protégées par de nombreux brevets, faisant de CEA Tech le premier déposant de brevets étendus à l'international.

CEA Tech est implanté dans plusieurs régions françaises, y compris Grenoble, Saclay, Bordeaux, Nantes, Lille, et Metz, afin de répondre aux besoins locaux des PME, ETI, et grands groupes.

Le CEA Tech a pour mission de développer et diffuser des technologies innovantes pour améliorer la compétitivité industrielle et répondre aux défis sociétaux. Ses activités principales incluent la recherche et le développement technologique dans des domaines tels que le numérique, la santé, l'énergie et le climat. CEA Tech se distingue par ses plateformes technologiques de pointe à Grenoble et Saclay, qui offrent des services de R&D, de prototypage et de validation de technologies.

3.3.2 Présentation de mon Institut : CEA-List

Le CEA-List, une unité de recherche technologique du CEA, se spécialise dans les systèmes numériques intelligents. Sa mission principale est de promouvoir l'innovation et de faciliter le transfert technologique vers l'industrie, renforçant ainsi la compétitivité des entreprises partenaires. Chaque année, le CEA-List collabore

avec plus de 200 entreprises, s'appuyant sur son expertise en intelligence artificielle, cybersécurité, et calcul quantique. Reconnue mondialement, l'institut publie régulièrement dans des revues internationales et dépose de nombreux brevets. Il gère 10 plateformes technologiques et 22 environnements de développement logiciel, répondant de manière flexible et confidentielle aux besoins industriels. En plus de ses projets collaboratifs, le CEA-List consacre 25

3.3.3 Présentation de mon laboratoire :

Au cœur du Plateau de Saclay (Île-de-France), l'institut CEA LIST focalise ses recherches sur les systèmes numériques intelligents. Porteurs d'enjeux économiques et sociétaux majeurs, ses programmes de R&D sont centrés sur les systèmes interactifs (intelligence ambiante), les systèmes embarqués (architectures, ingénierie logicielle et systèmes), les capteurs et le traitement du signal (contrôle industriel, santé, sécurité, métrologie).

Dédiés à la recherche technologique, les 700 ingénieurs-chercheurs et techniciens de l'Institut ont pour objectif de favoriser l'innovation et son transfert autour de partenariats industriels pérennes. La culture projet et l'excellence scientifique des équipes de l'Institut sont au cœur de cette ambition.

Au sein du CEA LIST, le Laboratoire de Simulation Interactive (LSI) développe une plateforme de simulation multi-physique interactive mettant en jeu un ou plusieurs utilisateurs en exploitant les technologies de Réalité Virtuelle (RV) et de Réalité Mixte (RM). Cette plateforme, dénommée XDE Physics, permet de simuler la manipulation et les interactions de l'ensemble des systèmes, pièces rigides, articulées ou déformables (câbles) directement sur les maquettes numériques. Elle permet également de valider des scénarios incluant l'opérateur pour étudier l'ergonomie du poste de travail par l'introduction de son avatar dans la simulation dynamique. Centrées sur les noyaux de simulation interactive, les activités de l'équipe vont jusqu'à la mise au point d'applicatifs, répondant aux contextes d'usage de ses partenaires industriels (manufacturing pour l'automobile et l'aéronautique, énergie, santé).

4 Travaux réalisés

4.1 Approche

Ce stage s'inscrit dans un contexte industriel visant à exploiter les technologies de reconstruction 3D pour la création de simulateurs haute-fidélité. L'objectif est de capturer des environnements réels et de les utiliser comme décors photoréalistes dans lesquels des objets 3D synthétiques peuvent être intégrés de manière cohérente. La méthode du 3D Gaussian Splatting (3DGS) [1], par sa capacité à générer des rendus temps réel de haute qualité à partir d'images, constitue une technologie de choix pour cette application.
Cependant, la méthode 3DGS originale, basée sur un processus de rastérisation, atteint ses limites face à des phénomènes optiques complexes. La problématique centrale de ce stage réside dans la faible qualité de rendu des scènes présentant des surfaces aux propriétés physiques particulières, notamment les effets spéculaires (reflets sur du métal), la réfraction (distorsion à travers le verre) et la transparence. Le modèle d'apparence de 3DGS, bien qu'efficace pour les surfaces diffuses, peine à modéliser la physique du transport de la lumière nécessaire pour représenter fidèlement ces matériaux.

Afin d'adresser cette limitation, une étude approfondie de l'état de l'art a été menée. Plusieurs approches récentes cherchant à dépasser les capacités de la rastérisation, principalement par l'introduction du lancer de rayons, ont été analysées et comparées. Des méthodes telles que 3DGRT [2], 3DGUT [3] ou encore RayGauss [4] ont été évaluées sur des jeux de données spécifiquement conçus pour mettre en exergue les difficultés liées aux reflets et à la transparence.
Au terme de cette analyse comparative, la méthode RayGauss [4] s'est clairement distinguée par la qualité visuelle supérieure de ses rendus. La solution technique idéale identifiée pour ce stage s'est donc articulée autour de deux axes majeurs directement inspirés de ces travaux :

1. Le remplacement du pipeline de rendu par rastérisation par une approche basée sur le lancer de rayons (*ray tracing*).
2. L'enrichissement de la représentation de la scène par l'ajout de *gaussiennes sphériques* pour mieux modéliser les reflets spéculaires.

La mise en œuvre de cette solution a été initiée par une phase d'exploration. Un prototype a d'abord été développé en C++ sur CPU afin d'implémenter et de valider la logique fondamentale du lancer de rayons au sein d'un nuage de gaussiennes. Cette étape a permis de maîtriser les concepts théoriques et algorithmiques de la première composante de la solution cible.
Cependant, le plan de développement a été adapté en cours de stage pour répondre à une demande client nécessitant une amélioration rapide et visible du rendu des surfaces réfléchissantes. L'effort s'est alors concentré sur l'implémentation de la composante la plus impactante et la plus rapide à intégrer dans l'écosystème logiciel existant. Plutôt que d'entreprendre la refonte complète du moteur pour y intégrer le lancer de rayons, la seconde partie des travaux a consisté à intégrer uniquement le concept de gaussiennes sphériques au sein du moteur de rendu par rastérisation du laboratoire, `3dgs-studio`. Cette approche pragmatique, accélérée par GPU, a permis d'améliorer la représentation des reflets sans modifier en profondeur l'architecture de rendu existante.
Les sections suivantes détaillent ces étapes clés dans leur ordre chronologique : l'analyse de l'état de l'art qui a défini la solution cible, le développement du prototype CPU explorant le lancer de rayons, et enfin l'implémentation pragmatique et ciblée des gaussiennes sphériques sur GPU pour répondre aux besoins du projet.

4.2 Première Partie : Etat de l'art

La méthode 3D Gaussian Splatting (3DGS) [1] a établi un nouveau standard pour la synthèse de vues temps réel. Son succès repose sur la représentation d'une scène par des gaussiennes 3D et sur un pipeline de rastérisation qui est à la fois rapide et entièrement différentiable. Le processus d'entraînement et de rendu, illustré en Figure 4, peut être décomposé en plusieurs étapes clés.

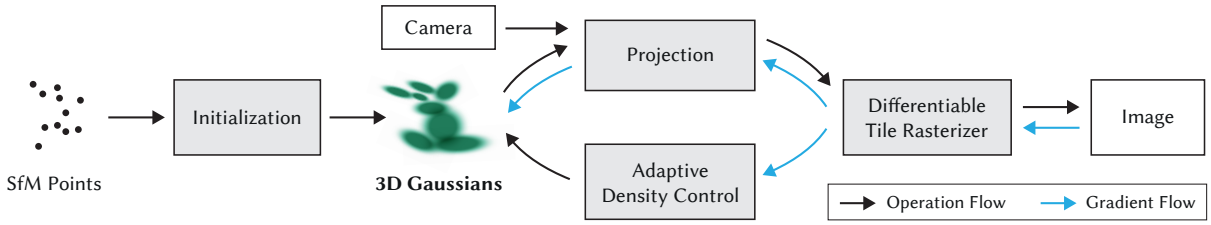


FIGURE 4 – Vue d'ensemble du pipeline de 3D Gaussian Splatting. À partir d'images et de poses de caméras obtenues par SfM, un ensemble de gaussiennes est optimisé via un rendu différentiable. Ce processus inclut un contrôle adaptatif de la densité pour affiner la représentation de la scène.

Initialisation à partir de Structure-from-Motion (SfM). Le point de départ de la méthode est un ensemble de N images d'une scène statique. Un algorithme de *Structure-from-Motion* (SfM), comme COLMAP, est d'abord appliqué à ces images. Cette étape produit deux éléments essentiels : les **poses de caméra** et un **nuage de points épars** qui fournit une première approximation de la géométrie. Chaque point de ce nuage est ensuite utilisé pour initialiser une gaussienne 3D.

Rendu volumétrique et optimisation différentiable. Le cœur de 3DGS est l'optimisation des paramètres des gaussiennes, qui s'appuie sur un processus de rendu volumétrique différentiable. Ce rendu peut être réalisé de deux manières principales. La méthode 3DGS originale utilise la **rastérisation** : les gaussiennes 3D sont triées par profondeur puis projetées sur le plan image sous forme d'ellipses 2D (*splats*) pour accumuler la couleur de chaque pixel. Une approche alternative, au cœur des travaux plus récents, est le **lancer de rayons**, où un rayon est lancé depuis la caméra pour chaque pixel et traverse séquentiellement les ellipsoïdes des gaussiennes sur son chemin.
Dans les deux cas, la couleur finale C d'un pixel est calculée par composition alpha ("alpha compositing") des N gaussiennes qui influencent ce pixel, triées d'avant en arrière, selon l'équation de rendu volumétrique :

$$C = \sum_{i=1}^{N} c_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j) \tag{1}$$

où c_i est la couleur de la i-ème gaussienne (évaluée dans la direction de la vue) et α_i est son opacité après projection. Le terme $\prod_{j=1}^{i-1}(1-\alpha_j)$ représente la **transmittance**, c'est-à-dire la quantité de lumière qui atteint la gaussienne i après avoir traversé les gaussiennes qui la précèdent.
La différentiabilité de cette équation est fondamentale : elle permet de calculer une fonction de perte (typiquement L1 + D-SSIM) entre l'image rendue C et l'image de référence, puis de rétropropager le gradient pour mettre à jour les paramètres de chaque gaussienne : position, covariance, opacité et couleur.

Contrôle adaptatif de la densité. Le nuage de points initial de SfM est souvent trop épars. Pour pallier ce manque, 3DGS met en œuvre un mécanisme de contrôle de densité adaptatif, illustré en Figure 5, pendant l'optimisation. Périodiquement, les gaussiennes sont analysées :

— **Densification :** Les gaussiennes situées dans des zones mal reconstruites (où le gradient de position est élevé) sont soit *clonées* (si elles sont petites), soit *divisées* en deux gaussiennes plus petites (si elles sont grandes). Cela permet d'ajouter des détails là où c'est nécessaire.

— **Élagage (*Pruning*) :** Les gaussiennes qui deviennent quasi-transparentes (opacité très faible) sont supprimées pour maintenir la compacité du modèle.

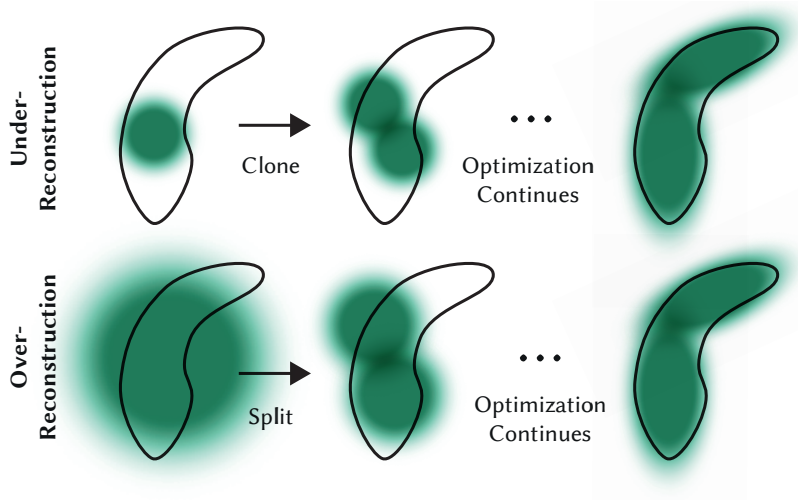


FIGURE 5 – Illustration du contrôle adaptatif de la densité. En haut, une gaussienne dans une zone sous-reconstruite est clonée. En bas, une gaussienne dans une zone sur-reconstruite est divisée en deux gaussiennes plus petites pour mieux capturer la géométrie. (Image adaptée de [1])

Modélisation de l'apparence et ses limites. L'apparence de chaque gaussienne dans la méthode 3DGS originale est modélisée à l'aide d'harmoniques sphériques (HS). Les harmoniques sphériques sont une base de fonctions définies sur la surface d'une sphère, analogues à la base de Fourier pour les signaux périodiques. Elles permettent de décomposer une fonction complexe — ici, la couleur émise dans chaque direction de l'espace — en une somme pondérée de fonctions de base plus simples.
Le degré 0 des HS correspond à une fonction constante sur la sphère ; il représente la composante diffuse de la couleur (*albedo*), indépendante de la direction d'observation. Les degrés supérieurs (1, 2, 3...) introduisent des variations directionnelles de plus en plus complexes. Chaque degré l étant composé de $2l + 1$ ordres m qui décrivent différentes orientations. Par convention, on se réfère uniquement au degré maximal, mais l'utilisation de tous les ordres correspondants est implicite. Le 3DGS original se limite ainsi typiquement au degré 3.
Cette représentation est très efficace pour modéliser des phénomènes de basse fréquence, comme l'éclairage diffus sur une surface mate. Cependant, elle est fondamentalement inadaptée à la représentation des phénomènes de haute fréquence. Un reflet spéculaire net, comme celui sur du chrome, est une distribution de lumière très concentrée dans une direction précise. Pour représenter un tel pic de lumière avec des harmoniques sphériques, il faudrait une somme d'un très grand nombre de coefficients (un degré très élevé), ce qui serait coûteux en mémoire et en calcul. En se limitant à un bas degré, les HS ne peuvent qu'approximer ce pic par une large tache floue, transformant un reflet net en un simple dégradé de couleur (voir Figure 6).

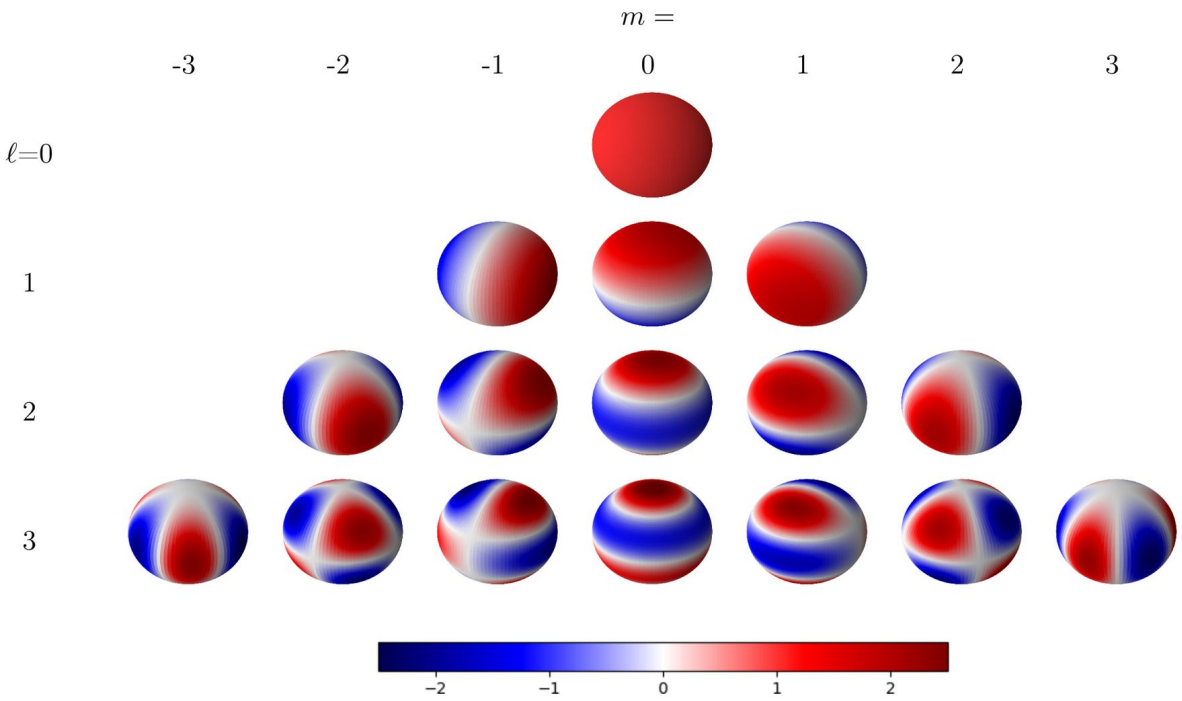


FIGURE 6 – Présentation des harmoniques sphériques Y_l^m jusqu'au degré $l = 3$. *(Source : planet-terre.ens-lyon.fr)*

... aux gaussiennes sphériques pour les hautes fréquences. Pour surmonter cette limitation, plusieurs travaux récents se sont tournés vers des représentations d'apparence plus expressives. La méthode **RayGauss** [4] se distingue particulièrement par l'introduction des **gaussiennes sphériques** (GS) en complément des harmoniques sphériques.
Une gaussienne sphérique est une fonction gaussienne définie directement sur la sphère. De son côté, elle est paramétrisée de manière plus intuitive : par une direction centrale, une concentration (qui contrôle la netteté du "lobe") et une amplitude (son intensité). Une seule gaussienne sphérique peut ainsi modéliser de manière compacte et précise un pic de lumière très directionnel. Elle est donc l'outil mathématique idéal pour représenter un reflet spéculaire. En pratique, l'apparence d'une surface est modélisée par un mélange : les harmoniques sphériques de bas ordre continuent de représenter la base diffuse, tandis qu'un ou plusieurs lobes de gaussiennes sphériques sont ajoutés pour capturer les reflets spéculaires.
Cette double représentation permet de gérer à la fois les basses fréquences (surfaces diffuses) et les hautes fréquences (reflets nets), offrant une fidélité visuelle nettement supérieure pour les matériaux complexes.

Vers une simulation physique : l'approche par lancer de rayons. Pour surmonter les limites du modèle d'apparence et de la rastérisation, une partie de la communauté scientifique s'est orientée vers le lancer de rayons (*ray tracing*). Contrairement à la rastérisation qui projette des ellipses 2D sur l'image, le lancer de rayons simule la trajectoire de la lumière à travers la scène. Lorsqu'un rayon traverse une gaussienne,

il est possible de réaliser une **intégration volumétrique** le long de son parcours. Ce calcul, bien que plus coûteux en temps de calcul, est physiquement plus juste et permet de déterminer plus précisément la couleur et l'opacité accumulées par le rayon.
Cette approche offre un autre avantage majeur : elle facilite l'intégration d'effets lumineux complexes et l'interaction avec d'autres types de géométries. Il devient ainsi plus aisé d'ajouter des objets maillés traditionnels dans la scène et de simuler des phénomènes physiques comme les **réflexions**, la **réfraction** à travers des matériaux transparents, ou encore le calcul des **ombres portées**.
Cette tendance se reflète dans plusieurs travaux récents. Certains, comme **3DGRT** [2], ont développé des algorithmes de lancer de rayons directs au sein des scènes gaussiennes. D'autres ont exploré des approches hybrides combinant la vitesse de la rastérisation avec la précision du lancer de rayons pour certains effets, comme **3DGUT** [3] ou **3DGS-DR** [5]. Enfin, des recherches se sont concentrées sur des aspects spécifiques, tels que la gestion des réflexions multiples (**IRGS** [6]) ou l'amélioration de l'éclairage d'environnement (**EnvGS** [7]).

Mise en place de l'environnement d'expérimentation. Avant toute analyse comparative, une étape préliminaire mais substantielle a été la mise en place des environnements d'exécution pour chaque méthode étudiée. Le déploiement de codes de recherche s'est révélé être un processus exigeant, les fichiers de configuration fournis (`environment.yaml`) étant souvent inopérants sur notre infrastructure cible en raison d'incompatibilités de dépendances. Une résolution manuelle de ces conflits a donc été nécessaire, en s'appuyant sur le gestionnaire d'environnements virtuels `conda` et le canal `conda-forge`.
L'intégration de bibliothèques externes non-Python, comme NVIDIA OptiX, a également requis une configuration système spécifique. Pour mener à bien ces installations, j'ai mobilisé l'infrastructure du laboratoire, incluant un poste de travail sous Linux et surtout la station DGX. Cette dernière, équipée de quatre GPU NVIDIA H100, a été un atout majeur pour lancer plusieurs entraînements en parallèle et accélérer la phase d'expérimentation. Un environnement de développement à distance a permis de piloter efficacement les codes sur le serveur.
Malheureusement, ces difficultés illustrent la fragilité des écosystèmes logiciels en recherche. Malgré de multiples tentatives, il n'a pas été possible de lancer l'expérimentation pour la méthode `EnvGS`. Une succession de problèmes d'installation, liés à l'environnement virtuel, à la version du pilote graphique et aux dépendances comme OptiX, a rendu son déploiement infructueux dans le temps imparti.

Analyse comparative et choix de la solution. Au cours de mon analyse, RayGauss s'est imposé comme la méthode la plus complète, car elle combine les deux innovations majeures : un pipeline de rendu entièrement basé sur le lancer de rayons volumétrique et un modèle d'apparence enrichi par les gaussiennes sphériques. Pour valider ce choix, une analyse comparative a été menée sur le jeu de données "Sac à main". Cette scène est particulièrement pertinente car elle présente des défis spéculaires variés : les **anneaux en métal** produisent des reflets nets de haute fréquence, tandis que la surface du **sac en cuir** offre des brillances plus larges et diffuses. Le tableau 1 présente les métriques de qualité d'image moyennes pour plusieurs méthodes de l'état de l'art.

TABLE 1 – Tableau comparatif pour le dataset "Sac à main" (métriques moyennes).

Méthode	**PSNR** ↑	**SSIM** ↑	**LPIPS** ↓	**Temps**
3DGS (base) [1]	30.327	0.941	0.116	00 :09 :12
Ray_Gauss [4]	32.232	0.962	0.060	08 :00 :00
3DGUT [3]	28.453	0.932	0.362	00 :40 :00
3DGS-DR [5]	28.064	0.922	0.267	03 :00 :00
IRGS [6]	18.795	0.663	0.440	02 :22 :52
SV-Raster [8]	30.368	-1	-1	00 :07 :33

L'analyse de ces résultats révèle un compromis clair. RayGauss surpasse qualitativement toutes les autres approches, mais au prix d'un temps d'entraînement prohibitif de huit heures. Alors que plusieurs méthodes de ce comparatif s'appuient déjà sur le lancer de rayons pour améliorer les résultats de la rastérisation de base, RayGauss se distingue en combinant un rendu volumétrique par lancer de rayons particulièrement poussé **avec** le modèle d'apparence avancé des gaussiennes sphériques. C'est cette **double complexité** qui explique vraisemblablement son avance en qualité, mais aussi son coût de calcul très élevé par rapport aux autres approches, y compris celles basées sur le lancer de rayons.
Cette observation a été déterminante pour la stratégie adoptée lors de ce stage. Constatant le coût de cette double approche, il a été décidé de ne pas réimplémenter l'intégralité de RayGauss. La solution pragmatique a été d'isoler ce qui semblait être sa contribution la plus significative à la qualité des reflets, le modèle d'apparence, et de l'intégrer au moteur de rendu par **rastérisation** existant. Comme le montreront les résultats de mes propres expérimentations plus loin, le surcoût lié à l'ajout des seules gaussiennes sphériques n'est que très peu impactant, permettant ainsi d'obtenir une part significative de l'amélioration qualitative sans en payer le prix en temps de calcul.
Pour compléter ces métriques quantitatives, la figure 7 présente une comparaison visuelle des rendus. Il est important de noter que les détails de haute fréquence des reflets spéculaires sont, par nature, difficiles à apprécier pleinement sur une image statique de taille réduite. La véritable supériorité de RayGauss se manifeste lors d'une navigation interactive dans la scène, où la stabilité et la netteté des reflets en mouvement deviennent plus évidentes.



(a) Vérité terrain (GT) (b) 3DGS [1] (c) RayGauss [4] (d) 3DGUT [3]

(e) 3DGS-DR [5] (f) IRGS [6] (g) SV-Raster [8]

FIGURE 7 – Comparaison visuelle sur la scène "Sac à main". Bien que les différences sur image fixe soient subtiles, on note la meilleure définition des reflets spéculaires de RayGauss (c) comparé à l'aspect plus flou de 3DGS (b). Cette observation corrobore les métriques du tableau 1.

En conclusion de cet état de l'art, l'analyse quantitative et visuelle a confirmé que la combinaison d'un pipeline basé sur le lancer de rayons et d'un modèle d'apparence enrichi par les gaussiennes sphériques est la voie la plus prometteuse. La méthode RayGauss s'est donc imposée comme la référence technique la plus pertinente pour atteindre notre objectif d'améliorer la restitution des matériaux à effets complexes, et a été retenue comme solution cible pour ce stage.

4.3 Seconde partie : Développement d'un prototype de lancer de rayons sur CPU

Conformément à l'objectif de surpasser les limites de la rastérisation, la première phase du stage a été consacrée au développement d'un prototype de rendu par lancer de rayons. Afin de maîtriser les concepts fondamentaux avant de considérer une implémentation sur GPU, il a été décidé de réaliser cette première version en C++ pour une exécution sur CPU. Cette section détaille les étapes de sa conception, depuis la gestion des données initiales jusqu'à l'implémentation d'un rendu volumétrique.

Prérequis et architecture logicielle. Pour l'architecture logicielle, j'ai adopté une structure inspirée de l'état de l'art, notamment de `3dgs-studio` et `RayGauss` [4]. Le code est scindé en deux parties : un script principal en Python pour la logique de haut niveau (chargement des données, gestion des scènes) et une bibliothèque C++ pour les calculs intensifs. La communication entre les deux est assurée par `pybind11`, un choix standard dans l'écosystème 3DGS qui facilite la séparation des responsabilités et prépare un portage futur vers des noyaux de calcul CUDA.

Initialisation et vérification des données. La première étape du travail a consisté à développer un module capable de charger les scènes pré-entraînées par 3DGS. Ces scènes sont généralement stockées dans un fichier au format `.ply`, qui contient l'ensemble des paramètres optimisés pour chaque gaussienne.

La structure de ce fichier est définie par un en-tête textuel, comme illustré dans le Listing 1, qui précède les données binaires. Cet en-tête déclare toutes les propriétés associées à chaque gaussienne.

```
ply
format binary_little_endian 1.0
element vertex 755228
property float x
property float y
property float z
property float nx
property float ny
property float nz
property float f_dc_0
property float f_dc_1
property float f_dc_2
property float f_rest_0
... (45 properties for f_rest)
property float f_rest_44
property float opacity
property float scale_0
property float scale_1
property float scale_2
property float rot_0
property float rot_1
property float rot_2
property float rot_3
end_header
```

Listing 1 – Exemple d'en-tête d'un fichier `.ply` utilisé par 3DGS. Il définit les 62 propriétés de chaque gaussienne.

Comme on peut le voir, chaque gaussienne (décrite comme un `vertex`) est définie par sa position (`x`, `y`, `z`), sa rotation sous forme de quaternion (`rot_0` à `rot_3`), ses facteurs d'échelle (`scale_0` à `scale_2`) et son opacité. L'apparence est modélisée par les coefficients d'harmoniques sphériques : les trois coefficients `f_dc` représentent la couleur de base (degré 0), tandis que les 45 coefficients `f_rest` encodent les variations directionnelles des degrés supérieurs.
Afin de valider le bon chargement de ces données, un premier rendu simple a été réalisé. En extrayant uniquement la position de chaque gaussienne et sa couleur de base (calculée à partir de ses coefficients `f_dc`), il a été possible de générer un nuage de points coloré. La comparaison visuelle de ce nuage avec les images d'origine a confirmé la bonne interprétation du fichier (voir Figure 8).



FIGURE 8 – Visualisation du nuage de points initial avec OpenCV. Chaque point correspond au centre d'une gaussienne, dont la couleur est déterminée par le coefficient de base de ses harmoniques sphériques. Cette étape a permis de valider le chargement des données de la scène.

Mise en place de la structure d'accélération. Pour interagir avec des primitives géométriques, mon prototype a besoin de traiter les gaussiennes non pas comme des nuages de densité, mais comme des objets solides avec une frontière définie. L'approche, reprise de l'algorithme de RayGauss, consiste à représenter chaque gaussienne par un **ellipsoïde de contour**. Cet ellipsoïde correspond à la surface où l'influence de la gaussienne devient négligeable, une limite définie par un seuil de visibilité, `SIGMA_THRESHOLD`.
Cette définition d'un ellipsoïde concret est cruciale pour deux étapes. La première est la construction d'une structure d'accélération, une **hiérarchie de volumes englobants** (en anglais, *Bounding Volume Hierarchy* ou BVH). Pour que cette structure puisse référencer nos primitives, chacune doit fournir sa boîte englobante alignée sur les axes (AABB). Le calcul de cette boîte consiste à trouver l'étendue maximale de l'ellipsoïde orienté le long des axes X, Y et Z du monde. Le seuil `SIGMA_THRESHOLD` joue ici un double rôle : il détermine la taille de l'ellipsoïde à englober et, si la gaussienne est quasi transparente, il réduit la taille de sa boîte à zéro, l'excluant ainsi des calculs pour optimiser le rendu.

```
1  // Extrait de la méthode de calcul de la BBox
2  BBox compute_gaussian_bbox() const {
3      // Calcul d'un facteur d'échelle basé sur la densité via le seuil
4      // Si la densité est trop faible, delta sera 0 et la BBox nulle.
5      float delta = std::logf((density / SIGMA_THRESHOLD) * (density / SIGMA_THRESHOLD))
       ;
6      delta = (delta < 0) ? 0.0f : std::sqrtf(delta);
7
8      // Calcul de l'étendue (extent) de l'ellipsoïde sur chaque axe du monde
9      // en projetant ses 3 axes locaux (L1, L2, L3) sur les axes du monde.
10     float ex = std::sqrtf(
```

```
        (scaling[0]*delta*L1[0])*(scaling[0]*delta*L1[0]) +
        (scaling[1]*delta*L2[0])*(scaling[1]*delta*L2[0]) +
        (scaling[2]*delta*L3[0])*(scaling[2]*delta*L3[0]));
    // ... calcul similaire pour ey et ez ...

    Vec3 extent(ex, ey, ez);
    return BBox(center - extent, center + extent);
}
```

Listing 2 – Construction des Bounding Boxes

Implémentation du lancer de rayons et de l'intersection. Le cœur du prototype a été l'implémentation de la fonction de lancer de rayons. Pour chaque pixel de l'image de sortie, un rayon est généré depuis la position de la caméra et traverse la scène. La seconde étape où l'ellipsoïde de contour est fondamental est le test d'intersection précis. Une fois que le BVH nous signale qu'un rayon intersecte l'AABB d'une gaussienne, ce test s'effectue contre le même ellipsoïde défini par `SIGMA_THRESHOLD`.
La stratégie adoptée consiste à transformer le rayon dans l'espace local de l'ellipsoïde (où ce dernier devient une simple sphère unité) plutôt que de transformer l'ellipsoïde lui-même, ce qui simplifie considérablement la résolution mathématique.

```
// Extrait de la méthode d'intersection
std::optional<std::pair<float, float>> intersect_gaussian(const Ray& ray) const {
    // 1. Définition de la taille de l'ellipsoïde via le seuil de densité
    float ratio = density / SIGMA_THRESHOLD;
    Vec3 scales = scaling * sqrtf(logf(ratio * ratio));
    // ...

    // 2. Transformation du rayon dans l'espace local de la gaussienne
    // où l'ellipsoïde devient une sphère unité centrée à l'origine.
    Vec3 O = ray.org - center;
    Vec3 O_ellipsis = /* M * */ O; // M est la matrice de transformation inverse
    Vec3 dir_ellipsis = /* M * */ ray.dir;

    // 3. Résolution de l'intersection analytique entre le rayon transformé et la sphère unité
    float b = -dot(O_ellipsis, D_ellipsis); // D_ellipsis est dir_ellipsis normalisé
    float c = dot(O_ellipsis, O_ellipsis) - 1;
    float disc = /* b*b - c (simplifié) */;

    if (disc > 1e-7f) {
        // Le rayon intersecte la sphère, on retourne les distances t_min et t_max
        return std::make_optional(std::make_pair(t_min, t_max));
    }
    return std::nullopt;
}
```

Listing 3 – Calcul d'intersection rayon/ellipsoïde

Avant de mettre en œuvre le rendu volumétrique complet, une étape de validation cruciale a été réalisée pour confirmer le bon fonctionnement de toute la chaîne géométrique. Un rendu simplifié a été produit : la couleur d'un pixel était déterminée uniquement par la *première* gaussienne intersectée par le rayon correspondant. La

couleur affichée était la couleur de base, indépendante de la vue, calculée à partir des coefficients de degré 0 des harmoniques sphériques.
L'objectif de ce test n'était pas de produire une image physiquement correcte, mais de fournir une confirmation visuelle claire que le pipeline géométrique — de la configuration de la caméra et la génération des rayons jusqu'à la traversée du BVH et la logique d'intersection rayon-ellipsoïde — fonctionnait comme attendu. Le résultat, présenté en figure 9, a validé la cohérence de ces calculs.

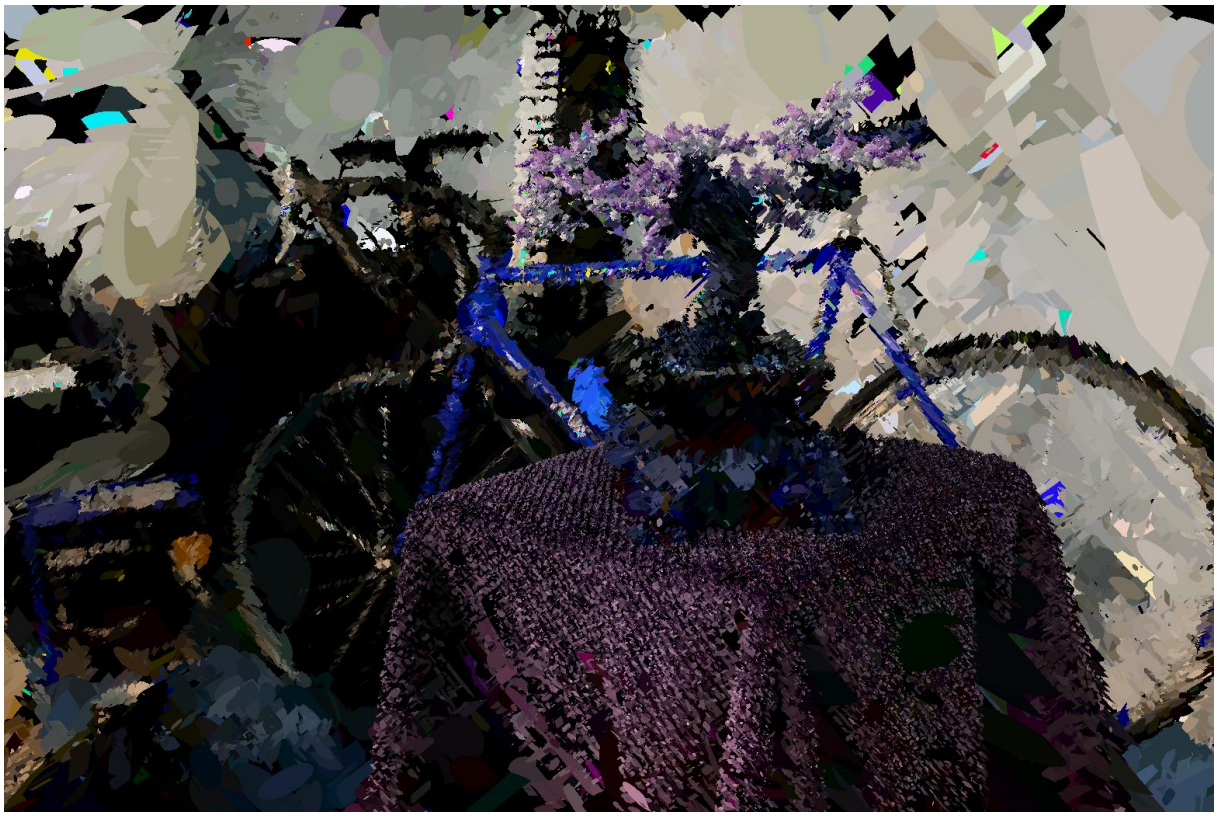


FIGURE 9 – Rendu de validation intermédiaire. Chaque pixel prend la couleur de base (harmonique sphérique de degré 0) de la première gaussienne intersectée. Cette étape a permis de valider la justesse des calculs du BVH et des intersections.

Implémentation du rendu *forward* pour la synthèse d'images. Dans le contexte des modèles neuronaux, on distingue deux processus fondamentaux : la passe *forward* et la passe *backward*.

- La **passe *forward*** (ou inférence) consiste à utiliser un modèle entraîné pour générer un résultat. Dans notre cas, il s'agit de synthétiser une nouvelle image à partir de la scène de gaussiennes optimisées. C'est un processus de synthèse.
- La **passe *backward*** (ou rétropropagation) est utilisée lors de l'entraînement. Elle calcule les gradients de la fonction de perte par rapport aux paramètres du modèle pour les mettre à jour. C'est un processus d'optimisation.

Ce chapitre se concentre sur l'implémentation du rendu **forward** par lancer de rayons.

Commissariat à l'énergie atomique et aux énergies alternatives
Institut List | Département Intelligence Ambiante et Systèmes Intéractifs
Service Intérations et Réseaux | Laboratoire de Simulation Intéractive
CEA Saclay - Nano-Innov - Bâtiment 861 - Point Courrier 173
91191 Gif-sur-Yvette Cedex
T. +33 (0)1 69 08 00 99
www-list.cea.fr
Établissement public à caractère industriel et commercial | RCS PARIS B 775 685 019

Choix d'un algorithme parallèle : la méthode par dalles (*slabs*). Bien que le prototype actuel ait été implémenté sur CPU, le choix de l'algorithme a été guidé par l'objectif d'une future parallélisation sur GPU. Le rendu volumétrique le long d'un rayon est par nature séquentiel, ce qui est peu efficace sur une architecture massivement parallèle.
Pour cette raison, la méthode par "dalles" (*slabs*), décrite dans `RayGauss` et illustrée en Figure 10, a été adoptée. Son principe est de découper la scène en tranches épaisses. Plutôt que de traiter chaque rayon individuellement de bout en bout, on lance un noyau de calcul pour traiter *tous les rayons* simultanément à travers *une seule dalle*. Pour chaque dalle, le BVH est interrogé en parallèle pour identifier les gaussiennes concernées. Cette approche par lots est parfaitement adaptée à l'exécution sur GPU.

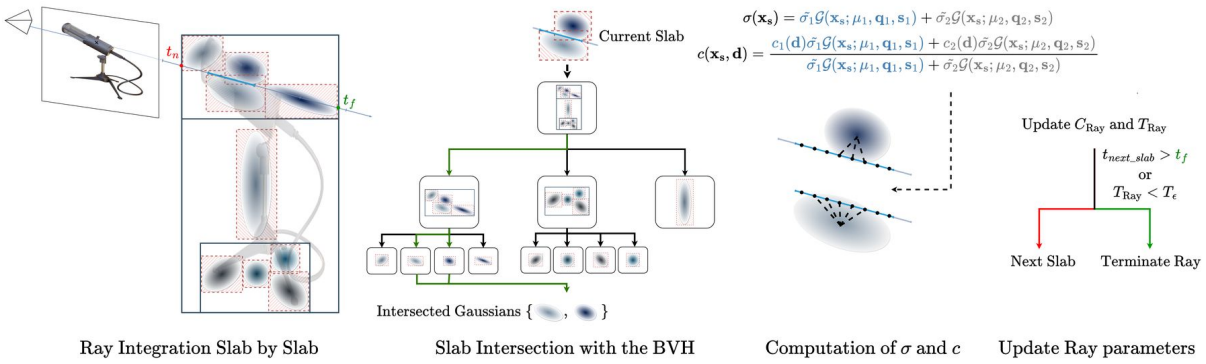


FIGURE 10 – Illustration du rendu par dalles (*slabs*) proposé par RayGauss. Un rayon est traité par dalles successives. Pour une dalle donnée, le BVH identifie les gaussiennes intersectées, dont les contributions sont ensuite accumulées pour mettre à jour la couleur et la transmittance du rayon.

L'implémentation de ce rendu se déroule en deux étapes principales pour chaque pixel.

Étape 1 : Accumulation des contributions par dalle. D'abord, la fonction `computeBufferForward` est appelée pour chaque segment de rayon traversant une dalle. Elle parcourt toutes les gaussiennes intersectant ce segment (`slabHits`). Pour plusieurs points d'échantillonnage discrets le long du segment, elle évalue la densité et la couleur apportées par chaque gaussienne et accumule ces contributions dans des tampons temporaires (`density_buffer` et `color_buffer`), comme le montre le Listing 4.

```
1  void computeBufferForward(/*...*/) {
2      // Pour chaque gaussienne intersectant le segment de rayon
3      for (const auto& hit : slabHits) {
4          Gaussian& gaussian = gaussians[hit.prim_id];
5          // ... (calcul de la couleur de la gaussienne, M, etc.)
6
7          // Pour chaque point d'échantillonnage dans le segment
8          for (int i=0; i < BUFFER_SIZE; i++){
9              Vec3 hit_sample = rayOrg + rayDir * t_sample[i];
10
11             // Évaluation de la densité de la gaussienne en ce point
12             float weight_density = evaluateGaussian(hit_sample, gaussian);
13
14             // Accumulation dans les tampons
```

```
15              if (weight_density > SIGMA_THRESHOLD) {
16                  density_buffer[i] += weight_density;
17                  color_buffer[i] += gaussian_color * weight_density;
18              }
19          }
20      }
21  }
```

Listing 4 – Étape 1 : Accumulation de la densité et de la couleur dans une dalle.

Étape 2 : Composition volumétrique le long du rayon. Ensuite, la fonction `colorBlending` prend le relais. Elle parcourt les tampons remplis, qui représentent la densité et la couleur agrégées à chaque pas le long du rayon. Elle applique alors la formule de composition volumétrique "alpha-blending" (voir équation 1) pour calculer la couleur finale du pixel, en mettant à jour la transmittance à chaque pas.

```
1  void colorBlending(/*...*/) {
2      // Remplir les tampons pour la dalle courante
3      computeBufferForward(/*...*/);
4
5      // Composer les valeurs accumulées dans les tampons
6      for(int i=0; i < BUFFER_SIZE; i++){
7          if (density_buffer[i] > 0) {
8              float alpha = 1.0f - expf(-density_buffer[i] * dt);
9              Vec3 color_sample = color_buffer[i] / density_buffer[i];
10
11             // Formule de composition "over"
12             ray_color += transmittance * alpha * color_sample;
13             transmittance *= (1.0f - alpha);
14         }
15     }
16 }
```

Listing 5 – Étape 2 : Composition des valeurs des tampons le long du rayon.

L'implémentation de ces deux fonctions a constitué l'aboutissement de ce prototype, permettant d'obtenir une image complète et physiquement plausible.

4.4 Validation des résultats par comparaison visuelle

Pour conclure le développement de ce prototype, il est essentiel de valider la justesse de son implémentation. Une première expérience, illustrée en Figure 11, met en évidence un principe fondamental : l'impact de la cohérence entre les données d'entraînement et l'algorithme de rendu.



(a) Rendu avec des données 3DGS (HS seuls)



(b) Rendu avec des données RayGauss (HS+GS)

FIGURE 11 – Illustration de l'incompatibilité des données d'entraînement. À gauche, notre prototype de lancer de rayons appliqué à des données entraînées pour la rastérisation 3DGS. À droite, le même prototype appliqué à des données entraînées avec RayGauss, incluant les gaussiennes sphériques.

L'image de gauche (a) montre le résultat de notre prototype de lancer de rayons sur une scène entraînée avec le pipeline 3DGS de base. Le rendu est visuellement dégradé. Cette différence ne provient pas d'une erreur d'implémentation, mais d'une incompatibilité fondamentale : les paramètres des gaussiennes ont été optimisés pour un rendu par rastérisation et ne sont donc pas adaptés à notre pipeline de lancer de rayons.
À l'inverse, l'image de droite (b) montre le rendu du même prototype sur une scène dont les paramètres ont été optimisés de bout en bout par un entraînement basé sur le lancer de rayons (celui de RayGauss). La qualité visuelle est immédiatement restaurée. Cette expérience démontre que la cohérence entre la méthode d'entraînement et la méthode de rendu est cruciale, et valide par la même occasion la justesse de notre implémentation : elle produit un résultat correct lorsqu'elle est utilisée avec des données entraînées de manière cohérente.
Ayant ainsi validé la justesse de l'implémentation sur une scène de référence, il est maintenant possible de généraliser cette validation en comparant les rendus du prototype à ceux de son implémentation de référence, `RayGauss`, sur un ensemble plus large de scènes. La Figure 12 présente cette comparaison côte à côte pour huit scènes différentes.

Commissariat à l'énergie atomique et aux énergies alternatives
Institut List | Département Intelligence Ambiante et Systèmes Intéractifs
Service Intérations et Réseaux | Laboratoire de Simulation Intéractive
CEA Saclay - Nanno-Innov - Bâtiment 861 - Point Courrier 173
91191 Gif-sur-Yvette Cedex
T. +33 (0)1 69 08 00 99
www-list.cea.fr
Établissement public à caractère industriel et commercial | RCS PARIS B 775 685 019



(a) Prototype (Ficus)



(b) Référence (Ficus)



(c) Prototype (Mic)



(d) Référence (Mic)



(e) Prototype (Chair)



(f) Référence (Chair)



(g) Prototype (Drums)



(h) Référence (Drums)



(i) Prototype (Hotdog)



(j) Référence (Hotdog)



(k) Prototype (Ship)



(l) Référence (Ship)



(m) Prototype (Materials)



(n) Référence (Materials)



(o) Prototype (Lego)



(p) Référence (Lego)

FIGURE 12 – Comparaison visuelle entre les rendus du prototype (à gauche de chaque paire) et les rendus de référence (à droite) pour l'ensemble des scènes.

Commissariat à l'énergie atomique et aux énergies alternatives
Institut List | Département Intelligence Ambiante et Systèmes Intéractifs
Service Intérations et Réseaux | Laboratoire de Simulation Intéractive
CEA Saclay - Nanno-Innov - Bâtiment 861 - Point Courrier 173
91191 Gif-sur-Yvette Cedex
T. +33 (0)1 69 08 00 99
www-list.cea.fr
Établissement public à caractère industriel et commercial | RCS PARIS B 775 685 019

Validation du prototype et orientation stratégique. On observe une très grande fidélité visuelle entre les deux ensembles d'images. Cette correspondance valide l'approche algorithmique choisie et confirme que le prototype CPU implémente avec succès les principes fondamentaux du rendu volumétrique par lancer de rayons.
L'analyse de la qualité des rendus, notamment la netteté des spécularités obtenues par RayGauss sur la scène du "Sac à main", a conduit à une prise de décision stratégique. Il est apparu que l'amélioration la plus significative pour les reflets provenait de l'utilisation des **gaussiennes sphériques**, bien plus que du pipeline de lancer de rayons lui-même, qui induisait un coût de calcul prohibitif.
Par conséquent, il a été décidé en se basant sur ces observations d'intégrer ce modèle d'apparence avancé directement au sein du moteur de rendu par **rastérisation** du laboratoire, `3dgs-studio`. Cette approche pragmatique a permis d'obtenir une amélioration tangible de la qualité des rendus, avec un surcoût en calcul bien plus faible que si le pipeline de lancer de rayons avait été porté sur GPU. La suite du stage s'est donc concentrée sur l'implémentation de cette solution à la fois sur la passe *forward* (inférence) et *backward* (entraînement) de ce moteur existant.

4.5 Troisième Partie : Intégration des Gaussiennes Sphériques dans le moteur de rastérisation

Comme établi à la fin de la section précédente, l'analyse des performances de l'état de l'art a conduit à une décision stratégique : plutôt que de poursuivre le développement d'un pipeline complet par lancer de rayons, l'effort a été concentré sur l'intégration de sa composante la plus prometteuse pour le rendu des spécularités — les gaussiennes sphériques (GS) — au sein du moteur de rendu par rastérisation existant du laboratoire, `3dgs-studio`. Cette section détaille les étapes de cette intégration, depuis les modifications de l'architecture logicielle jusqu'à l'analyse des résultats finaux.

Architecture et intégration du modèle. L'intégration a débuté par la duplication de la classe principale `GSplatModel`, renommée en `SGSplatModel`. Cette nouvelle classe hérite de toutes les fonctionnalités de base (gestion de la scène, densification, sauvegarde) et a été étendue pour gérer les trois nouveaux paramètres des gaussiennes sphériques :

- `sph_gauss_features` : les coefficients de couleur des lobes spéculaires.
- `bandwidth_sharpness` : la concentration (netteté) de chaque lobe.
- `lobe_axis` : la direction centrale de chaque lobe.

De nouveaux optimiseurs avec des taux d'apprentissage spécifiques (`_lr`) ont été ajoutés aux fichiers de configuration pour ces attributs, en se basant sur les valeurs recommandées par RayGauss. La classe a également été adaptée pour charger et sauvegarder des fichiers `.ply` compatibles avec le format de RayGauss, ce qui assure l'interopérabilité et la persistance des modèles entraînés.

Modification du pipeline de rendu et des noyaux CUDA. Le cœur de la modification réside dans la passe *forward*. Le principe est additif : la couleur finale est la somme de la contribution des harmoniques sphériques (basses fréquences) et de celle des gaussiennes sphériques (hautes fréquences). Le pipeline de rendu de `3dgs-studio`, basé sur PyTorch, a été modifié comme suit :

```
# 1. Calcul de la couleur diffuse et speculaire basse frequence via les HS
colors = spherical_harmonics(sh_degree, dirs, shs, masks=masks)
```

```
3
4 # 2. Ajout de la couleur speculaire haute frequence via les GS
5 colors += spherical_gaussians(num_sph_gauss, dirs, sph_gauss_features,
6                               bandwidth_sharpness, lobe_axis, masks=masks)
```

Listing 6 – Addition des contributions des HS et des GS dans le rendu forward.

Pour des performances optimales, la fonction `spherical_gaussians` s'appuie sur des noyaux CUDA personnalisés. Le Listing 7 illustre le noyau *forward*, qui calcule la couleur apportée par les GS pour une direction de vue donnée.

```
1 // Dans la fonction device sg_features_to_color(...)
2 float result = 0.0f;
3 float inorm = rsqrtf(dir.x * dir.x + dir.y * dir.y + dir.z * dir.z);
4 vec3 dir_normalized = dir * inorm;
5
6 #pragma unroll
7 for (uint32_t k = 0; k < num_sg_to_use; ++k) {
8     vec3 lobe_axis_(lobe_axis[k * 3 + 0], lobe_axis[k * 3 + 1], lobe_axis[k * 3 + 2]);
9     scalar_t dot_product_axis = glm::dot(dir_normalized, lobe_axis_);
10    scalar_t gaussian = __expf(bandwidth_sharpness[k] * (dot_product_axis - 1.0f));
11    result += gaussian * sg_features[k * 3 + c];
12 }
13 colors[c] = result;
```

Listing 7 – Extrait du noyau CUDA forward calculant la contribution en couleur des GS.

De manière symétrique, la passe *backward* requiert son propre noyau personnalisé pour calculer les gradients des nouveaux paramètres, comme l'exige le mécanisme de différentiation automatique de PyTorch. Ce noyau, illustré dans le Listing 8, implémente la rétropropagation du gradient de la couleur (`v_colors`) vers les trois paramètres des GS.

```
1 // Dans la fonction device sg_features_to_color_vjp(...)
2 // ...
3 scalar_t dot_dir_lobe_minus1 = glm::dot(dir_normalized, lobe_axis_) - 1.0f;
4 scalar_t sph_gauss_weight = __expf(dot_dir_lobe_minus1 * sharpness);
5 scalar_t common_grad_term = glm::dot(v_colors_local, sph_gauss_feat_) *
      sph_gauss_weight;
6
7 // Gradient par rapport aux features de couleur
8 v_sg_features[k*3+0] = v_colors_local.x * sph_gauss_weight; // ...pour R, G, B
9
10 // Gradient par rapport a la nettete
11 v_bandwidth_sharpness[k] = common_grad_term * dot_dir_lobe_minus1;
12
13 // Gradient par rapport a l'axe du lobe
14 scalar_t common_sharpness_term = common_grad_term * sharpness;
15 v_lobe_axis[k*3+0] = dir_normalized.x * common_sharpness_term; // ...pour x, y, z
```

Listing 8 – Extrait du noyau CUDA backward calculant les gradients des paramètres des GS.

Pour ces deux noyaux, et contrairement aux harmoniques sphériques où chaque canal de couleur peut être traité indépendamment, les calculs de gradients des GS sont interdépendants. Il a donc été décidé d'utiliser **un seul thread par point** pour traiter les trois canaux de couleur simultanément, assurant ainsi la cohérence des calculs.

Résultats et analyse comparative. Pour évaluer l'efficacité de notre approche, une analyse qualitative a été menée, en commençant par la preuve la plus directe de l'amélioration des spécularités. La Figure 13 compare la composante spéculaire isolée sur la scène du "Bonsai", en opposant le rendu avec les harmoniques sphériques (HS) seules à celui de notre méthode avec les gaussiennes sphériques (GS).



(a) Spéculaire (HS seuls) (b) Spéculaire (HS + 7 GS)

FIGURE 13 – Comparaison de la composante spéculaire isolée sur la scène "Bonsai". Les HS seules (a) ne peuvent produire qu'une large nappe de couleur diffuse. Notre méthode (b) génère des reflets nets et structurés.

Cette comparaison est sans équivoque. La composante spéculaire des harmoniques sphériques seules se résume à une large tache floue, confirmant leur incapacité à modéliser des phénomènes de haute fréquence. En revanche, notre méthode produit des reflets nets et précis sur le socle verni, démontrant que les gaussiennes sphériques remplissent parfaitement leur rôle en capturant les détails des spécularités.
Cet ajout fondamental se traduit directement sur la qualité du rendu final. La Figure 14 illustre l'impact sur la scène du "Sac à main", en comparant notre modèle (HS de degré 2 + 7 GS) à la même base sans les GS.



(a) HS seuls (degré 2) (b) Notre méthode (HS + 7 GS)

FIGURE 14 – Étude d'ablation sur la scène "Sac à main". L'ajout des 7 gaussiennes sphériques (b) permet de restituer des reflets sur les anneaux métalliques bien plus nets et réalistes que le modèle de base avec seulement les HS de degré 2 (a).

L'amélioration est particulièrement visible sur les anneaux en métal. Alors que le modèle avec seulement les HS produit des reflets diffus, notre approche parvient à capturer des spécularités bien définies, se rapprochant de la qualité attendue sur une surface métallique.
Enfin, pour démontrer la généralité de cette amélioration, la Figure 15 présente des comparaisons sur d'autres scènes aux matériaux variés.



(a) Bonsai (HS seuls)



(b) Bonsai (HS + 7 GS)



(c) Bicycle (HS seuls)



(d) Bicycle (HS + 7 GS)



(e) Garden (HS seuls)



(f) Garden (HS + 7 GS)

FIGURE 15 – Comparaisons visuelles grand format. Pour chaque paire, la version de gauche utilise les HS seuls, celle de droite notre méthode avec HS+GS. Les améliorations sont notables sur le socle verni du bonsaï, le banc en bois du vélo, ainsi que sur le centre de table en verre et la surface de la table de la scène du jardin.

Pour chaque scène, notre méthode améliore la restitution des surfaces réfléchissantes : le vernis du socle du bonsaï, le bois du banc et les surfaces en verre de la table de jardin montrent tous des brillances plus réalistes et mieux définies.

Analyse quantitative : surcoût en stockage et en calcul. Au-delà de l'amélioration qualitative, il est essentiel d'évaluer l'impact de cette nouvelle approche sur les ressources. Le premier coût mesurable est celui du stockage. L'ajout des gaussiennes sphériques augmente la quantité de données à stocker pour chaque primitive. Concrètement, si notre implémentation réduit le nombre de coefficients d'harmoniques sphériques (passant du degré 3 au degré 2), elle ajoute pour chaque gaussienne les 49 paramètres flottants de ses 7 lobes spéculaires (21 pour la couleur, 7 pour la netteté et 21 pour l'axe).
Cette augmentation du nombre de paramètres par gaussienne se traduit par une hausse notable de la taille des fichiers `.ply` finaux, comme le détaille le tableau 2.

TABLE 2 – Comparaison de la taille des fichiers `.ply` finaux.

Dataset	Taille (HS seuls)	Taille (HS + SG)	Augmentation
Truck	269.7 Mo	426.3 Mo	+ 58%
Sac à main	253.1 Mo	379.1 Mo	+ 50%
Bonsai	455.9 Mo	838.8 Mo	+ 84%
Musée	723.8 Mo	1.1 Go	+ 52%
Garden	1.3 Go	2.1 Go	+ 62%
Twingo	1.4 Go	2.1 Go	+ 50%
Bicycle	2.0 Go	3.0 Go	+ 50%

On observe une augmentation moyenne de la taille des fichiers de l'ordre de 50 à 80%, un surcoût non négligeable mais qui reste justifiable au vu de l'amélioration de la fidélité visuelle.
Finalement, l'analyse des temps d'entraînement sur plusieurs scènes (Tableau 3) confirme que cette amélioration qualitative s'obtient avec un surcoût de calcul maîtrisé, validant ainsi la pertinence de l'approche pragmatique adoptée.

TABLE 3 – Comparaison des temps d'entraînement totaux entre la méthode de base et notre implémentation.

Dataset	Temps (HS seuls)	Temps (HS + SG)	Surcoût
Truck	6m 35s	8m 25s	**+ 28 %**
Sac à main	58m 46s	59m 35s	**+ 1.4 %**
Bonsai	57m 25s	1h 15m 18s	**+ 31 %**
Twingo	1h 28m 49s	1h 39m 51s	**+ 12 %**
Bicycle	1h 59m	2h 7m 53s	**+ 7.5 %**
Garden	2h 34m 4s	2h 53m 1s	**+ 12 %**
Musée	4h 4m 41s	6h 9m 26s	**+ 51 %**

L'intégration des gaussiennes sphériques a donc permis d'atteindre l'objectif principal du stage : améliorer de manière significative et efficace la restitution des matériaux complexes, en obtenant une part substantielle de l'amélioration qualitative de l'état de l'art sans en payer le prix en temps de calcul prohibitif.

5 Conclusion et Perspectives

Conclusion. Le travail réalisé durant ce stage s'est articulé en deux phases distinctes, reflétant une démarche de recherche pragmatique. Une première phase d'exploration, constituant un travail de fond, a consisté à développer un prototype de lancer de rayons sur CPU. Cette étape a permis de valider la maîtrise des algorithmes complexes du rendu volumétrique et a servi de fondation à la décision stratégique qui a suivi. Forte de l'analyse des coûts de l'état de l'art, la seconde phase, qui constitue la contribution principale de ce stage, a consisté à intégrer les gaussiennes sphériques directement dans le moteur de rastérisation GPU du laboratoire, `3dgs-studio`, en développant pour cela les noyaux CUDA nécessaires pour les passes *forward* et *backward*.
Les résultats de cette seconde phase ont démontré le succès sans équivoque de l'approche. L'analyse visuelle, notamment par l'isolation de la composante spéculaire, a prouvé la capacité des gaussiennes sphériques à modéliser des reflets nets et précis, là où les harmoniques sphériques seules échouent. De plus, cette amélioration qualitative significative a été obtenue avec un surcoût en temps d'entraînement qui reste maîtrisé. La contribution principale de ce stage est donc la démonstration qu'une amélioration ciblée du modèle d'apparence est une stratégie très efficace, permettant d'obtenir une part substantielle de la qualité des méthodes de lancer de rayons sans en subir le coût de calcul prohibitif.

Perspectives : vers un rendu hybride et un rééclairage dynamique. Bien que l'intégration des gaussiennes sphériques dans un moteur de rastérisation se soit avérée une stratégie efficace, le potentiel du lancer de rayons reste immense et peut être exploré selon deux axes distincts et complémentaires.
Une première piste, orientée vers la passe ***forward*** (inférence), consisterait à utiliser le lancer de rayons pour intégrer des objets maillés 3D au sein du décor capturé par 3DGS. Cette approche, démontrée par NVIDIA dans 3DGUT, permettrait à ces objets synthétiques d'interagir de manière physiquement plausible avec la scène, en projetant des ombres portées ou en créant des reflets et des réfractions réalistes sur les surfaces gaussiennes.
Une seconde piste, plus fondamentale et axée sur la passe ***backward*** (entraînement), serait d'employer le lancer de rayons pour améliorer la qualité de la reconstruction 3DGS elle-même. Pour des scènes contenant déjà des surfaces complexes, comme des miroirs ou des vitres, des rayons secondaires pourraient être lancés durant l'optimisation pour mieux entraîner les splats à rendre compte de ces interactions lumineuses. Cette approche ouvre la voie à des applications de ***relighting*** avancées. En effet, une fois la géométrie et l'apparence de la scène correctement découplées, il deviendrait possible de modifier les matériaux, de déplacer les sources lumineuses ou de rééclairer dynamiquement la scène. Pour de tels phénomènes, un pipeline de lancer de rayons est indispensable.
Cependant, le coût de calcul de ces approches, tant pour l'entraînement que pour l'inférence, pose un défi majeur, notamment dans un contexte de réalité virtuelle. En VR, un temps d'inférence extrêmement bas est crucial pour maintenir 60 à 90 images par seconde sur deux vues stéréoscopiques et ainsi garantir une expérience fluide. Dans cette optique, des solutions hybrides comme 3DGUT, qui combinent une rastérisation rapide de la scène principale avec des lancers de rayons secondaires pour les interactions complexes, apparaissent comme une voie d'avenir très prometteuse. Faute de temps, ces pistes n'ont pu être explorées durant ce stage, mais elles constituent une suite logique et passionnante aux travaux réalisés.

Références

[1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), July 2023.

[2] Nicolas Moenne-Loccoz, Ashkan Mirzaei, Or Perel, Riccardo de Lutio, Janick Martinez Esturo, Gavriel State, Sanja Fidler, Nicholas Sharp, and Zan Gojcic. 3d gaussian ray tracing : Fast tracing of particle scenes. *ACM Transactions on Graphics and SIGGRAPH Asia*, 2024.

[3] Qi Wu, Janick Martinez Esturo, Ashkan Mirzaei, Nicolas Moenne-Loccoz, and Zan Gojcic. 3dgut : Enabling distorted cameras and secondary rays in gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.

[4] Hugo Blanc, Jean-Emmanuel Deschaud, and Alexis Paljic. Raygauss : Volumetric gaussian-based ray casting for photorealistic novel view synthesis, 2024.

[5] Ye Keyang, Hou Qiming, and Zhou Kun. 3d gaussian splatting with deferred reflection. 2024.

[6] Chun Gu, Xiaofei Wei, Zixuan Zeng, Yuxuan Yao, and Li Zhang. Irgs : Inter-reflective gaussian splatting with 2d gaussian ray tracing. In *CVPR*, 2025.

[7] Tao Xie, Xi Chen, Zhen Xu, Yiman Xie, Yudong Jin, Yujun Shen, Sida Peng, Hujun Bao, and Xiaowei Zhou. Envgs : Modeling view-dependent appearance with environment gaussian. *arXiv preprint arXiv :2412.15215*, 2024.

[8] Cheng Sun, Jaesung Choe, Charles Loop, Wei-Chiu Ma, and Yu-Chiang Frank Wang. Sparse voxels rasterization : Real-time high-fidelity radiance field rendering. *ArXiv*, abs/2412.04459, 2024.

6 Annexes

`https://julesbaji.github.io`